Revisiting ContextAgent: A Reproducibility Study of Context-Aware Proactive LLM Agents under Resource Constraints

Anonymous authors
Paper under double-blind review

Abstract

Proactive Large Language Model (LLM) agents decide on their own when to offer assistance, which requires them to evaluate context continuously and makes their inference cost a first-order concern. We reproduce *ContextAgent* (Yang et al., 2025b), a context-aware proactive agent evaluated on *ContextAgentBench*, using the authors' released code and data split, and then examine whether its results hold under common efficiency techniques: parameter-efficient fine-tuning choices, base-model quantisation and pruning. The central claim of the original work, that supervised fine-tuning with persona context and thought traces clearly outperforms in-context learning baselines, is confirmed for all three base models, and the in-context baselines reproduce closely. However, the fine-tuned models score substantially *higher* than reported, and the gap depends on a discrepancy we identified between the paper (learning rate 10^{-4}) and the released training configuration (5×10^{-5}): with the paper's stated learning rate, tool-calling F1 rises from the reported 0.626/0.647/0.645 to 0.781/0.689/0.779 for Llama-3.1-8B, DeepSeek-R1-7B and Qwen2.5-7B. Beyond reproduction, we find that 4-bit NF4 quantisation of the base model roughly halves inference memory while keeping tool-calling F1 within 1.1 points of the corresponding 16-bit model for the two instruction-tuned models (2–3 points with the released default configuration), at the cost of 30–50% higher latency with the bitsandbytes runtime. Unstructured pruning up to 50% sparsity largely preserves task performance but yields no efficiency gain with dense kernels, whereas structured pruning of feed-forward layers reduces memory and latency but degrades performance quickly beyond 10% sparsity. We release code and configurations.

1 Introduction

Most LLM-based agents are reactive: they produce an answer or perform an action only after receiving a prompt from the user. A different line of work studies proactive agents, which use available context to decide whether assistance is needed and may act without being explicitly instructed to do so (Lu et al., 2024). This changes the role of the agent from answering requests to taking the initiative, which also affects how users perceive and work with the system (Diebel et al., 2024). It also changes the practical cost of running the system. A reactive agent performs inference only on user interaction, whereas a proactive agent runs continuously or at regular intervals to monitor context, decide whether intervention is needed and possibly call external tools. A proactive agent may therefore perform well on a benchmark while still being too costly to operate in practice.

ContextAgent (Yang et al., 2025b) is a recent framework for context-aware proactive LLM agents that combines sensory context, persona information, proactive prediction and tool calling. It is accompanied by *ContextAgentBench*, a benchmark with 1,000 samples across nine daily scenarios and 20 tools, and by a public implementation. The original work focuses on capability: it shows that supervised fine-tuning of 7–8B models with persona context and thought traces outperforms in-context learning baselines on proactive prediction and tool calling. The cost of maintaining this behaviour is not evaluated.

This paper is a reproducibility study of *ContextAgent*. We first reproduce the main results on *ContextAgentBench* with the authors' code and data split. We then test whether the conclusions hold, and at what cost, when the fine-tuned reasoner is made cheaper to train and run with widely used efficiency techniques: Low-Rank Adaptation (LoRA) configuration choices, base-model quantisation (LLM.int8 and 4-bit 4-bit NormalFloat (NF4)) and pruning (unstructured, semi-structured and structured). Our contributions are:

- **Reproduction of the main claims.** The in-context learning baselines reproduce within a few points of the reported values, and the supervised *ContextAgent* configuration outperforms them for all three base models, confirming the central claim (Section 6.1).
- **A paper–code discrepancy and its effect.** The paper reports a learning rate of 10^{-4}, whereas the released training configuration uses 5×10^{-5}. Both settings yield higher fine-tuned scores than reported, and the paper's setting yields the highest; for example, Llama-3.1-8B reaches a tool-calling F1 of 0.781 against the reported 0.626 (Table 2).
- **Robustness under efficiency constraints.** We quantify the cost–performance trade-off of LoRA rank and target modules, quantisation and pruning for the *ContextAgent* reasoner, measuring training and inference memory, latency and throughput alongside the original task metrics (Section 6.2).

The study is limited to *ContextAgent* as the framework and *ContextAgentBench* as the evaluation setting, and it covers the context-aware reasoner only; the perception stage (vision-language model and speech recognition) is outside the cost measurements.

2 Related Work

2.1 Proactive LLM Agents

Most LLM agents are reactive systems, where the model acts after receiving an explicit user prompt. ReAct (Yao et al., 2023) combines reasoning and action in an iterative process, which is relevant for tool use and multi-step decision making, but the agent is still given a task or observation to respond to. A proactive agent must additionally decide whether assistance should be provided at all.

Lu et al. (2024) frame proactivity as a prediction problem in which the model learns when proactive suggestions are likely to be accepted by users. Yang et al. (2025a) study proactivity in a deployed setting, where sensory context is gathered through on-demand perception and used together with LLM reasoning to provide proactive assistance on AR glasses. *ContextAgent* (Yang et al., 2025b) combines sensory context, persona information, proactive prediction and tool calling, and evaluates both whether the agent should act and how it should act. These systems primarily target proactive capability; their cost of operation is not the main subject of evaluation.

2.2 Efficient Model Adaptation and Compression

Parameter-efficient fine-tuning methods such as LoRA reduce training cost by freezing the base model and training a small set of adapter parameters (Hu et al., 2021). Quantised Low-Rank Adaptation (QLoRA) extends this approach by quantising the frozen model during training (Dettmers et al., 2023), and LLM.int8 provides 8-bit inference with outlier handling (Dettmers et al., 2022). Pruning removes parameters or structures from an existing model (Cheng et al., 2024; Sun et al., 2024; Fang et al., 2023), while Knowledge Distillation (KD) trains a smaller student model to imitate a larger teacher (Hinton et al., 2015). Surveys report these as the most common approaches for reducing the cost of LLMs (Gun Il Kim, 2025; Pierre V. Dantas, 2025), but they are usually evaluated on reactive tasks. Whether they preserve the behaviour a proactive agent needs, namely calibrated proactive scores, a fixed structured output format and valid tool arguments, is the question this paper addresses for *ContextAgent*.

2.3 Reproducibility Studies

Independent reproduction of published machine learning results, as promoted by the ML Reproducibility Challenge and the reproducibility tracks of information retrieval venues, regularly reveals undocumented choices and discrepancies between papers and released code. Following the ACM terminology, our study is a *reproduction* (different team, same artefacts) of the *ContextAgentBench* results, extended with a *generalisation* to resource-constrained settings.

3 Background

This section introduces the proactive agent setting, the *ContextAgent* framework and the *ContextAgentBench* benchmark.

3.1 Proactive LLM Agents

A proactive LLM agent differs from a reactive agent in the decision problem it solves. A reactive agent receives an explicit instruction from the user and produces an output. A proactive agent instead uses available context to decide whether assistance is needed and may act without being directly prompted (Lu et al., 2024). A simple way to view such a system is as operating on a sequence of contextual states $c_1, c_2, \ldots, c_t$, where each c_t may include observations from the environment, user-related information, and the current system state.

At each configurable sampling step t, the agent computes $f(c_t) = (p_t, a_t)$, where p_t represents the estimated need for assistance and a_t is the planned action. If $p_t < \theta$, where θ is an adjustable threshold, the agent stays passive. If $p_t \geq \theta$ the agent begins assistance. In practice a_t may be a direct response, a tool call or a sequence of tool calls followed by a response.

This also helps explain why proactive agents have a different cost profile than reactive agents. A reactive system performs inference only when prompted, while a proactive system may need to evaluate context repeatedly even when the user is not requesting it. The total cost is therefore shaped not only by the complexity of a single inference step, but also by how often the decision process is triggered and by how much computation is needed to turn these observations into usable context.

3.2 ContextAgent

This general idea is made concrete in *ContextAgent* (Yang et al., 2025b), which combines sensory perceptions and persona context in order to predict the need for proactive assistance and call external tools when needed. The agent task is formalised as:

$$(T, PS, TC, R) = A(S, P) \tag{1}$$

where A is the agent, S denotes sensory perceptions and P denotes persona context. The outputs are thought traces T, a proactive score PS, a planned tool chain TC and a final response R. Proactive assistance is triggered only when $PS \geq \theta$.

This maps directly to the general framework $f(c_t) = (p_t, a_t)$ from Section 3.1, where $c_t = (S, P)$, $p_t = PS$, and $a_t = (TC, R)$.

The framework has two main parts: proactive-oriented context extraction and context-aware proactive reasoning. The first part is tasked with turning raw sensory input into a representation that is easier for the LLM to use. The second part then uses this representation together with persona context to predict whether assistance is needed and which tools should be used.

The sensory input S is viewed as multimodal and is described as including egocentric video, audio and smartphone notifications. From this input, the framework constructs a context representation $C = (C_V, C_A, N)$, where C_V is visual context extracted from video, C_A is audio context extracted from speech, and N denotes textual information from smartphone notifications that is incorporated directly into the final context representation.

For video, *ContextAgent* uses a vision language model with in-context learning to generate visual context C_V. For audio, it uses speech recognition to generate audio context C_A. Persona context P is also used, since the same sensory situation can imply different needs depending on a user's preferences, identity and past behaviour.

After extracting context, the framework applies a context-aware reasoner:

$$(T, PS, TC) = A_S(C, P) \tag{2}$$

where A_S is an LLM that takes sensory context C and persona context P as input and produces thought traces, a proactive score and a planned tool chain. The planned tool chain is a sequence of tool calls with arguments. A simple way to represent this is:

$$TC = \big((t_i, a_i)\big)_{i=1}^{m} \qquad t_i \in \mathcal{T} \tag{3}$$

where $\mathcal{T}$ is the set of available tools, t_i is the selected tool and a_i is the corresponding argument set.

The reasoning model is trained with supervised fine-tuning on a dataset $\mathcal{D}_{\text{SFT}} = \{(X_i, T_i, Y_i)\}_{i=1}^{n}$ where X_i contains the sensory context C_i and persona context P_i, T_i is the thought trace, and Y_i contains the proactive score PS_i and planned tool chain TC_i.

During inference, if the predicted score satisfies $PS \geq \theta$, the planned tools are then executed in sequence. The outputs are then combined with the sensory context, persona context and thought traces to generate the final response.

3.3 ContextAgentBench

ContextAgentBench is the benchmark used in the *ContextAgent* paper to evaluate context-aware proactive agents. Each sample contains seven parts:

$$(S, C, P, T, PS, TC, R) \tag{4}$$

where S is the sensory perception, C is the contextual summary, P is the persona context, T is the thought trace, PS is the proactive score, TC is the planned tool chain, and R is the final response. This means that the benchmark does not only ask if an agent should act. It also includes the context used for the decision, the reasoning trace, the planned use of tools and the final response.

The benchmark is thus a dataset

$$\mathcal{B} = \{(S_i, C_i, P_i, T_i, PS_i, TC_i, R_i)\}_{i=1}^{N} \tag{5}$$

with $N = 1000$. The original paper also introduces *ContextAgentBench-Lite*, with 300 human-verified samples paired with raw sensor data such as egocentric video and audio; we do not use it. The proactive score is defined on a five-point scale, with $PS_i \in \{1, 2, 3, 4, 5\}$, where 1 means that no proactivity is required and 5 means a high level of proactivity.

For non-proactive cases, where $PS_i \in \{1, 2\}$, both the planned tool chain and final response are set to $TC_i = \texttt{None}$ and $R_i = \texttt{None}$. For the proactive cases, the tool chain may contain several tool calls. If $\mathcal{T}$ denotes the set of tool types, then it can be written as:

$$TC_i \in \mathcal{T}^{\leq 5} \cup \{\texttt{None}\} \tag{6}$$

since the benchmark contains 20 tool types and each sample may involve up to five tools.

The benchmark is built in two stages. First, the paper creates 200 human-written seed examples from proactive and non-proactive situations in daily life. These samples include contextual summaries, persona information when needed, proactive scores, thought traces and planned tool chains for proactive cases. Each sample is cross reviewed for plausibility and format. Second, the benchmark is expanded through an automated diversification that generates additional samples with LLMs. Generation is conditioned either on

the scenario or on the proactive score. The resulting samples are checked by annotators for rationality and a script to verify the data format and tool arguments.

The released benchmark comes with a fixed split of 600 training and 400 test samples, which we use throughout. Because it provides labels for proactive scores, tool chains and arguments, the original task setting can be reproduced before introducing measures of computational cost.

4 Methods

This section describes the reproduction protocol and the efficiency techniques applied to the *ContextAgent* reasoner.

4.1 Reproduction Protocol

We reproduce the main *ContextAgentBench* experiment of Yang et al. (2025b) (their Table 1) for the three base models reported there: `Llama-3.1-8B-Instruct`, `Qwen2.5-7B-Instruct` and `DeepSeek-R1-7B`. We use the released code, prompts, data split and evaluation scripts; deviations are listed in Section 5.3. We test the following claims:

- **C1** Supervised fine-tuning with thought traces and personas (*ContextAgent*) outperforms all in-context learning baselines on both proactive prediction and tool calling.
- **C2** Adding persona context and thought traces to in-context learning (ICL-All) improves over Vanilla ICL.
- **C3** The reported absolute scores can be obtained with the released artefacts.

All models are evaluated with the original metrics and evaluation code: Proactive Prediction Accuracy (Acc-P), Missed Detection rate (MD), False Detection rate (FD) and Root Mean Square Error (RMSE) for proactive prediction, and precision, recall, F1-score and Argument Accuracy (Acc-Args) for tool calling (Section 5.6).

4.2 Selected Efficiency Techniques

We select techniques that are widely used to make LLMs cheaper to train and run. Surveys identify quantisation, pruning, knowledge distillation and parameter-efficient fine-tuning as the most common approaches for reducing computational cost and memory usage while maintaining model performance (Gun Il Kim, 2025; Pierre V. Dantas, 2025). These techniques are well established in reactive settings; we assess whether their benefits carry over to the *ContextAgent* reasoner, which must produce calibrated proactive scores, a fixed structured output format and valid tool arguments. Knowledge distillation into 1–1.5B student models was also attempted; it is reported only briefly as a negative result in Section 7.3.

4.3 Low-Rank Adaptation (LoRA)

ContextAgent already fine-tunes its reasoner with LoRA (rank 8, all linear layers). We therefore do not compare LoRA against full fine-tuning, but study how the LoRA configuration (rank, target modules, learning rate) and base-model quantisation affect task performance and cost.

LoRA freezes the pre-trained weights and injects trainable rank-decomposition matrices into each targeted layer of the *Transformer* (Hu et al., 2021). For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is constrained to a low-rank decomposition $W_0+\Delta W = W_0+BA$, where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and $r \ll \min(d,k)$. The forward pass computes

$$h = W_0x + \Delta Wx = W_0x + BAx. \tag{7}$$

This reduces the number of trainable parameters and therefore the optimiser-state memory, while performing comparably to full fine-tuning on many tasks (Hu et al., 2021). The rank r and the set of targeted weight matrices are the main capacity choices.

4.3.1 Quantisation and QLoRA

QLoRA trains LoRA adapters on top of a quantised, frozen base model (Dettmers et al., 2023). Base-model quantisation compresses the frozen weights into lower-precision data types, reducing the memory required to load the model during both training and inference. Since only the adapters are trained, aggressive quantisation of the base weights does not directly affect their optimisation. We use two quantisation levels, each with 16-bit adapters: 8-bit LLM.int8 and 4-bit NF4.

4-bit NormalFloat (NF4). NF4 is a 4-bit data type designed for zero-mean, normally distributed weights: it maps weights to 4-bit bins with equal expected probability mass, minimising quantisation error under that distributional assumption (Dettmers et al., 2023). This allows 7–8B models to be fine-tuned under tight memory limits while maintaining performance close to 16-bit baselines.

8-bit LLM.int8. LLM.int8 stores weights in 8-bit precision and preserves task performance by isolating outlier features, which are processed in 16-bit precision, while the vast majority of matrix multiplications are performed in 8-bit (Dettmers et al., 2022). Compared to NF4, it trades smaller memory savings for higher numerical fidelity.

4.4 Pruning

Pruning removes parameters or computational structures that contribute little to predictive performance. Unlike LoRA, which adds low-rank updates alongside the frozen weights, pruning directly modifies the original network. Let $\boldsymbol{\theta}$ denote the parameters of a model and $\mathbf{m} \in \{0,1\}^{|\boldsymbol{\theta}|}$ a binary pruning mask. The pruned model is $f(x; \mathbf{m} \odot \boldsymbol{\theta})$, and the objective is to minimise performance degradation under a given sparsity constraint (Cheng et al., 2024). For LLMs, pruning can reduce memory and computation at inference time, which could enable proactive agents on constrained hardware. Pruning strategies are commonly divided into unstructured pruning, which removes individual parameters, and structured pruning, which removes entire parameter groups such as channels, neurons or attention heads.

4.4.1 Unstructured Pruning

Pruning performance depends on how well the importance of weights is estimated. Importance scores are computed using a calibration dataset that reflects the inputs seen during deployment. This data is used to collect statistics such as activation magnitudes, allowing data-aware pruning without full retraining. We take the calibration data from the *ContextAgentBench* training split so that it reflects the task distribution.

Unstructured pruning removes weights independently without enforcing any pattern, allowing the model to retain the most important parameters at a given sparsity level. As a result, it typically retains more accuracy than more constrained pruning methods at the same sparsity, but it does not reduce memory or latency unless sparse kernels are used.

Among data-aware methods, Pruning by **W**eights **and A**ctivations (WANDA) (Sun et al., 2024) is an effective one-shot pruning approach. It estimates the importance of each weight by combining its value with the activation statistics of its input feature.

Consider a linear layer with weight matrix $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$. Given a calibration dataset, the input activations are collected into a matrix $\mathbf{X} \in \mathbb{R}^{(N \times L) \times C_{\text{in}}}$, where N is the batch size and L is the sequence length. The importance score for each weight w_{ij} is defined as:

$$S_{ij} = |w_{ij}| \cdot \|\mathbf{X}_j\|_2, \tag{8}$$

where $\mathbf{X}_j$ is the j-th column of $\mathbf{X}$, representing the activation of the j-th input feature.

Weights with the lowest importance scores are removed. This approach extends magnitude-based pruning by also considering how often a feature is used. By aggregating activations across the calibration data, the method captures usage patterns that are important in real tasks.

To provide a baseline, WANDA is compared with standard magnitude pruning, which uses only the size of weights. WANDA typically performs better at higher sparsity levels, where magnitude pruning may remove weights that are still important during inference.

These importance scores are extended to the semi-structured setting by enforcing N:M sparsity constraints. In this setup, each weight matrix is divided into blocks of M parameters, and only the N most important weights are kept in each block.

This semi-structured design balances accuracy and hardware efficiency. Results for the 2:4 and 4:8 sparsity patterns are reported in Appendix C.

4.4.2 Structured Pruning

Torch-Pruning is employed for structured pruning (Fang et al., 2023), a DepGraph-based framework that removes structurally coupled parameter groups while preserving model consistency. Pruning is applied post-training to a pretrained model, targeting primarily the feed-forward Multilayer Perceptron (MLP) layers, while attention head pruning is disabled to preserve architectural stability.

Unlike unstructured pruning, structured pruning operates on groups of parameters that are interdependent across layers. The DepGraph framework explicitly models these dependencies and ensures that coupled parameters are pruned jointly, enabling consistent removal of neurons or hidden dimensions without breaking the computational graph. As a result, pruning is performed at the group level rather than on individual parameters.

To estimate group importance, two criteria are evaluated within the same pruning framework. The first, `GroupMagnitudeImportance`, ranks parameter groups by the aggregated L2 norm of their weights, where the L2 norm captures the group's overall magnitude. The second, `ActivationImportance`, uses activation statistics collected from a calibration dataset through forward passes, reflecting how frequently and strongly each group contributes to the model's outputs.

Equal pruning ratios are applied across MLP layers, while attention heads remain unpruned, allowing for the isolation of the impact of structured sparsity in feed-forward components. This choice is made based on preliminary experiments in which attention head pruning revealed severe degradation in model performance, even at low sparsity levels. Consequently, head pruning is disabled in the final configuration, and structured pruning is applied exclusively to feed-forward MLP layers.

4.4.3 Pruning Shortcomings and Recovery Phase

Pruning modifies the model's optimisation by removing parameters or structural components that contribute to learned representations. As a result, even importance-aware pruning strategies often introduce accuracy degradation, distribution shifts in intermediate activations, and reduced calibration, particularly at higher sparsity levels. These effects are more pronounced in structured pruning, where removing entire heads or hidden dimensions disrupts inter-layer dependencies. To mitigate these issues, a recovery phase is commonly applied after pruning (Cheng et al., 2024), consisting of Supervised Fine-Tuning (SFT) on task-specific data. This recovery step allows the model to adapt to the induced sparsity through LoRA-based fine-tuning, partially restoring performance while preserving the efficiency gains introduced by pruning. In our experiments, all pruned models underwent the same recovery procedure to ensure a fair comparison across pruning strategies and sparsity regimes.

5 Experimental Setup

5.1 Data and Configurations

All experiments use the released *ContextAgentBench* split of 600 training samples (`cab_train_sft`) and 400 test samples (`cab_test`) (Yang et al., 2025b). Each sample contains the following fields:

- *Context information:* visual, audio and smartphone-notification context extracted from wearable sensors.
- *Personas:* a description of the user in the specific scenario.
- *Thoughts:* the reference thought trace.
- *Proactive score:* the amount of proactivity needed, from 1 (none) to 5 (high).
- *Tools:* the tool chain, including arguments, that the agent should call.
- *Response:* the agent's final free-text response.

The combinations of these fields define the methods in Table 1, following the original paper. *Vanilla ICL* uses the context information only; *CoT* adds thought traces; *ICL-P* adds personas; and *ICL-All* adds both. *ContextAgent* is the supervised configuration: the model is fine-tuned with LoRA adapters on all linear layers using sensory context, thought traces and personas, which makes it the fine-tuned counterpart of *ICL-All*.

5.2 Implementation Details

All fine-tuning uses the LLaMA-Factory configuration released with *ContextAgent*, with a sequence cut-off of 2048 tokens, 5 epochs, a cosine learning-rate schedule with 10% warm-up, bfloat16 mixed precision, gradient checkpointing and an effective batch size of 8 (per-device batch size 2, 4 gradient-accumulation steps). The default LoRA configuration is rank $r = 8$, $\alpha = 2r$, all linear layers, and learning rate 5×10^{-5}, as in the released code. Experiments ran on a single NVIDIA L4 GPU with 24 GB of memory. The software environment used PyTorch 2.7.0, Transformers 4.52.1, PEFT 0.15.2, LLaMA-Factory 0.9.3.dev0, and bitsandbytes 0.45.1. Quantised runs used the bitsandbytes backend for 4-bit quantisation. Generation used top-p/top-k sampling with $p = 0.7$ and $k = 50$, temperature 0.95, one beam, and a maximum of 1024 new tokens. For each configuration we report the result from the single completed training run ($k = 1$); because no independent repeated training runs were retained, between-run variation in tool-calling F1 is not estimable. As a rough guide to statistical uncertainty, with 400 test samples the 95% binomial confidence interval has a half-width of about ± 0.027 for an Acc-P of 0.92 and about ± 0.048 for an Acc-Args of 0.6. We therefore do not interpret differences of a few hundredths between configurations.

5.3 Deviations from the Original Setup

- **Learning rate discrepancy.** The ContextAgent paper reports an SFT learning rate of 10^{-4} (Section 5.1), whereas the released LLaMA-Factory training configuration files (`llama3_lora_sft.yaml`, `qwen_lora_sft.yaml`) specify 5×10^{-5}. Rather than assuming an error, we treat this as a paper–code discrepancy and evaluate both settings: we adopt the released configuration (5×10^{-5}) for our baseline reproduction and evaluate the paper's stated value (10^{-4}) in a separate learning-rate experiment (Table 2). Both settings outperform the reported numbers, with 10^{-4} yielding the highest tool-calling F1 across instruction-tuned models.
- **Hardware and batch size.** The paper reports training on eight A6000 GPUs. The released configuration uses per-device batch size 1 with 8 gradient-accumulation steps; if it was run data-parallel on eight GPUs, the effective batch size was 64 rather than 8, corresponding to eight times fewer optimiser steps. We train on a single GPU with an effective batch size of 8.
- **Configuration search.** The released split has no validation set. The LoRA sweeps in Section 6.2 are therefore evaluated on the test split and should be read as a sensitivity analysis; the "best" configurations are optimistic estimates.

5.4 Quantisation Settings

Quantised runs load the base model with bitsandbytes before injecting 16-bit adapters. The 8-bit runs use LLM.int8 (Dettmers et al., 2022); the 4-bit runs use NF4 with double quantisation (Dettmers et al., 2023).

The trainable adapters are kept in bfloat16. All other parameters are identical to the corresponding 16-bit run.

5.5 Pruning Settings

Pruning is evaluated on `Llama-3.1-8B-Instruct`, the strongest model in our reproduction, at sparsity ratios of 10%, 30% and 50% for unstructured and structured pruning, and with 2:4 and 4:8 patterns for semi-structured pruning (Appendix C). WANDA and magnitude pruning are compared for unstructured and semi-structured sparsity, while structured pruning uses group magnitude and activation-based importance. Data-aware criteria use 200 calibration samples from the training split with a fixed sequence length of 2048 tokens.

Evaluations immediately after pruning showed severe degradation at moderate and high sparsity, so all pruned models undergo the same LoRA-based recovery fine-tuning with the default configuration before evaluation. Prior to evaluation, the recovery LoRA adapters are merged into the pruned base weights via `merge_and_unload()`; the pruning mask is not re-applied afterwards, meaning evaluation is conducted with the adapter updates permanently incorporated into the weights. Pruned weights are stored in dense format, so unstructured and semi-structured sparsity is not exploited by the inference kernels.

5.6 Metrics

All configurations are evaluated on the 400 test samples with the original evaluation code.

Proactive prediction. Acc-P measures whether the predicted need for proactive assistance (proactive score above the threshold) matches the ground truth. The MD is the fraction of samples where assistance was needed but not predicted, and the FD is the fraction where assistance was predicted but not needed. RMSE measures the error of the predicted proactive score on the 1–5 scale.

Tool calling. Precision, recall and F1-score compare predicted and ground-truth tool names as sets. Acc-Args additionally requires the tool arguments to be correct: a sample with any incorrect argument counts as incorrect, and missed proactive actions are penalised (Yang et al., 2025b).

Costs. Training cost is reported as duration and peak and average GPU memory. Inference cost is reported as peak and average GPU memory (GB), average, minimum and maximum batch latency (s) and throughput (samples/s), with a batch size of 1.

6 Results

We first report the reproduction of the original results (Section 6.1) and then the efficiency experiments (Section 6.2).

6.1 Reproduction of the Original Results

Table 1 reports all methods with the released configuration, and Table 2 compares the reproduced ICL-All and *ContextAgent* results with the values reported by Yang et al. (2025b).

Table 1: Reproduction on *ContextAgentBench* (400 test samples) with the released configuration (*ContextAgent*: LoRA $r = 8$, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	Vanilla ICL	0.730	0.225	0.045	2.066	0.569	0.514	0.525	0.246
	CoT	0.713	0.242	0.045	2.007	0.568	0.503	0.519	0.259
	ICL-P	0.777	0.165	0.058	2.007	0.633	0.544	0.567	0.374
	ICL-All	0.755	0.212	0.033	1.935	0.602	0.545	0.558	0.294
	ContextAgent	**0.918**	**0.040**	0.038	**1.119**	**0.763**	**0.739**	**0.735**	**0.595**
DeepSeek-R1-7B	Vanilla ICL	0.627	0.315	0.058	2.711	0.470	0.445	0.451	0.129
	CoT	0.632	0.302	0.065	2.645	0.471	0.450	0.455	0.153
	ICL-P	0.710	0.212	0.077	2.387	0.514	0.467	0.479	0.152
	ICL-All	0.720	0.237	0.043	2.393	0.515	0.488	0.494	0.173
	ContextAgent	**0.873**	**0.048**	0.070	**1.365**	**0.625**	**0.622**	**0.609**	**0.368**
Qwen2.5-7B-Ins	Vanilla ICL	0.800	0.165	0.035	2.079	0.583	0.554	0.558	0.298
	CoT	0.782	0.193	0.025	2.095	0.563	0.530	0.538	0.249
	ICL-P	0.880	0.072	0.048	1.726	0.568	0.546	0.548	0.283
	ICL-All	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent	**0.915**	**0.038**	0.045	**1.136**	**0.734**	**0.719**	**0.708**	**0.551**

Table 2: Reported (Yang et al., 2025b) versus reproduced results. *Code*: released configuration (learning rate 5×10^{-5}); *paper*: learning rate 10^{-4} as stated in the original paper. Both use LoRA $r = 8$ on all linear layers.

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	ICL-All (reported)	0.757	0.229	0.012	1.872	0.631	0.565	0.582	0.270
	ICL-All (ours)	0.755	0.212	0.033	1.935	0.602	0.545	0.558	0.294
	ContextAgent (reported)	0.874	0.030	0.095	1.408	0.660	0.627	0.626	0.448
	ContextAgent (ours, code)	0.918	0.040	0.038	1.119	0.763	0.739	0.735	0.595
	ContextAgent (ours, paper)	0.940	0.033	0.023	0.943	0.802	0.791	0.781	0.661
DeepSeek-R1-7B	ICL-All (reported)	0.704	0.268	0.027	2.540	0.545	0.510	0.518	0.103
	ICL-All (ours)	0.720	0.237	0.043	2.393	0.515	0.488	0.494	0.173
	ContextAgent (reported)	0.888	0.027	0.085	1.319	0.676	0.648	0.647	0.468
	ContextAgent (ours, code)	0.873	0.048	0.070	1.365	0.625	0.622	0.609	0.368
	ContextAgent (ours, paper)	0.900	0.048	0.053	1.220	0.706	0.704	0.689	0.505
Qwen2.5-7B-Ins	ICL-All (reported)	0.867	0.088	0.044	1.721	0.635	0.577	0.591	0.301
	ICL-All (ours)	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent (reported)	0.894	0.013	0.091	1.264	0.672	0.644	0.645	0.459
	ContextAgent (ours, code)	0.915	0.038	0.045	1.136	0.734	0.719	0.708	0.551
	ContextAgent (ours, paper)	0.925	0.032	0.033	0.955	0.805	0.774	0.779	0.661

C1: fine-tuning outperforms in-context learning (reproduced). For all three models, *ContextAgent* achieves the highest Acc-P, the lowest MD and RMSE, and the best tool-calling scores of all methods (Table 1). The only exception is the FD, where some ICL baselines are lower, as in the original paper. For Llama and Qwen the margin over ICL-All is considerably larger than reported: tool-calling F1 improves by 0.177, 0.115 and 0.125 for Llama, DeepSeek and Qwen with the released configuration, against 0.044, 0.129 and 0.054 in the original paper.

C2: personas and thought traces help in-context learning (reproduced). ICL-All improves over Vanilla ICL in Acc-P, F1-score and Acc-Args for all three models. As in the original paper, adding thought traces alone (CoT) does not consistently help.

C3: reported absolute scores (partially reproduced). The in-context baselines reproduce closely: Acc-P is within 0.02 and F1-score within 0.025 of the reported ICL-All values. The fine-tuned models, however,

differ substantially. With the released configuration, Llama and Qwen exceed the reported tool-calling F1 by 0.109 and 0.063, while DeepSeek falls 0.038 below it. With the learning rate stated in the paper, all three models exceed the reported values, by 0.155 (Llama), 0.042 (DeepSeek) and 0.134 (Qwen) in F1 and by 0.037–0.213 in Acc-Args. The error profile also differs: the reported models have a low MD and a high FD (0.085–0.095), whereas our models are balanced (FD 0.023–0.070). A plausible explanation for the higher scores is the number of optimiser steps: with eight data-parallel GPUs and the released accumulation settings, the original runs would have used an effective batch size of 64 and about 50 optimiser steps, against about 375 in our runs (Section 5.3). We could not verify this with the released material. In either case, the qualitative conclusions of the original paper hold, and the released code yields a stronger *ContextAgent* than reported.

Cost of the reproduced models. Table 13 in Appendix B reports the training cost of the reproduced *ContextAgent* models: all three peak at about 21 GB of GPU memory and train in 45–58 minutes. Table 3 reports their inference cost, which serves as the reference for the efficiency experiments.

Table 3: Inference cost of the reproduced *ContextAgent* models (16-bit, released configuration).

		Costs					
Method	Model	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
ContextAgent	Llama-3.1-8B-Ins	15.6	15.6	9.0	4.0	21.7	0.1115
	DeepSeek-R1-7B	14.8	14.8	8.7	2.4	23.1	0.1148
	Qwen2.5-7B-Ins	14.8	14.8	8.3	1.7	22.6	0.1206

The three models need 14.8–15.6 GB at inference and process one sample in 8.3–9.0 s on average.

6.2 Efficiency under Resource Constraints

All experiments in this section use the *ContextAgent* configuration with thought traces and personas. Unless stated otherwise, the LoRA configuration is the released default: rank $r = 8$, all linear layers, learning rate 5×10^{-5}. Because all sweeps are evaluated on the test split (Section 5.3), they are a sensitivity analysis rather than a model selection procedure.

6.2.1 LoRA Configuration

Table 4: LoRA rank (all linear layers, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Rank	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	8	0.918	0.040	0.038	1.119	0.763	0.739	0.735	0.595
	16	0.935	0.033	0.033	1.033	0.803	0.795	0.782	0.659
	32	**0.941**	**0.030**	**0.032**	1.004	**0.806**	**0.797**	**0.787**	**0.685**
	64	0.935	0.033	0.033	**0.937**	0.802	0.793	0.782	0.683
DeepSeek-R1-7B	8	0.873	0.048	0.070	1.365	0.625	0.622	0.609	0.368
	16	0.883	0.048	0.060	1.295	0.658	0.661	0.646	0.433
	32	0.898	0.045	0.058	1.258	0.694	0.697	0.678	0.490
	64	**0.910**	**0.042**	**0.048**	**1.203**	**0.728**	**0.739**	**0.713**	**0.530**
Qwen2.5-7B-Ins	8	0.915	0.038	0.045	1.136	0.734	0.719	0.708	0.551
	16	0.923	0.038	0.040	1.075	0.774	0.742	0.739	0.618
	32	**0.940**	**0.032**	**0.025**	**0.927**	**0.798**	**0.782**	**0.770**	**0.632**
	64	0.935	0.032	0.035	0.987	0.795	0.771	0.769	0.631

Table 4 presents the effect of LoRA rank on task performance across all three models, with all other hyperparameters fixed to their baseline values. For `Llama-3.1-8B-Instruct`, the F1-score rises from 0.735 at $r = 8$ to 0.787 at $r = 32$ and then plateaus (0.782 at $r = 64$). `Qwen2.5-7B-Instruct` follows the same pattern, with F1-scores of 0.708, 0.739, 0.770 and 0.769. `DeepSeek-R1-7B` improves at every rank, from 0.609 at $r = 8$ to 0.713 at $r = 64$, without clear saturation within the tested range. Differences between $r = 32$ and $r = 64$ for Llama and Qwen are within run-to-run uncertainty.

Table 5: LoRA target modules ($r = 8$, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Target modules	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	All linear	**0.918**	**0.038**	**0.038**	**1.119**	**0.763**	**0.739**	**0.735**	**0.595**
	Attention only	0.900	0.065	0.045	1.261	0.723	0.707	0.699	0.512
	MLP only	0.905	0.038	0.058	1.223	0.731	0.714	0.706	0.556
DeepSeek-R1-7B	All linear	0.873	0.048	0.070	1.365	0.625	0.622	0.609	0.369
	Attention only	0.852	0.082	0.065	1.471	0.629	0.620	0.608	0.380
	MLP only	**0.885**	**0.048**	**0.065**	**1.301**	**0.689**	**0.687**	**0.673**	**0.489**
Qwen2.5-7B-Ins	All linear	**0.915**	**0.038**	**0.038**	**1.124**	**0.734**	**0.719**	**0.709**	**0.557**
	Attention only	0.878	0.050	0.058	1.312	0.643	0.629	0.620	0.512
	MLP only	0.915	0.047	0.038	1.137	0.732	0.715	0.708	0.553

Table 5 reports the effect of restricting LoRA adapters to specific architectural components, evaluated at the baseline rank and learning rate. For `Llama-3.1-8B-Instruct` and `Qwen2.5-7B-Instruct`, adapting all linear layers gives the best results (F1-score 0.735 and 0.709), and adapting only the attention projections gives the worst (0.699 and 0.620). For Qwen, adapting only the MLP layers is as good as adapting all linear layers (0.708). `DeepSeek-R1-7B` behaves differently: MLP-only adaptation (F1-score 0.673, Acc-Args 0.489) clearly outperforms both all-linear (0.609, 0.369) and attention-only adaptation (0.608, 0.380).

Table 6: Learning rate (LoRA $r = 8$, all linear layers).

		Proactive Predictions				Tool Calling			
Model	LR	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	2.0×10^{-5}	0.925	0.035	0.035	1.033	0.788	0.768	0.761	0.642
	5.0×10^{-5} (code)	0.918	0.040	0.038	1.119	0.763	0.739	0.735	0.595
	1.0×10^{-4} (paper)	**0.940**	**0.033**	**0.023**	**0.943**	**0.802**	**0.791**	**0.781**	**0.661**
DeepSeek-R1-7B	2.0×10^{-5}	0.883	0.053	0.065	1.273	**0.720**	**0.721**	**0.704**	**0.527**
	5.0×10^{-5} (code)	0.873	0.048	0.070	1.365	0.625	0.622	0.609	0.368
	1.0×10^{-4} (paper)	**0.900**	**0.048**	**0.053**	**1.220**	0.706	0.704	0.689	0.505
Qwen2.5-7B-Ins	2.0×10^{-5}	0.923	0.035	**0.033**	**0.987**	0.795	0.764	0.769	0.652
	5.0×10^{-5} (code)	0.915	0.038	0.045	1.136	0.734	0.719	0.708	0.551
	1.0×10^{-4} (paper)	0.925	**0.032**	**0.033**	0.955	**0.805**	**0.774**	**0.779**	**0.661**

Table 6 compares three learning rates: 2×10^{-5}, the released 5×10^{-5} and the paper's 10^{-4}. For Llama and Qwen, 10^{-4} gives the best tool-calling F1-score (0.781 and 0.779, against 0.735 and 0.708 with the released value). For DeepSeek, 2×10^{-5} is best (0.704) and 10^{-4} is close (0.689). For all three models the released value of 5×10^{-5} is the weakest of the three settings. The response is therefore not monotonic in the learning rate within this grid, and the gap to 5×10^{-5} is larger than the expected run-to-run variation. Since 10^{-4} is the largest value tested, we cannot exclude that larger learning rates perform better still.

6.2.2 Quantisation

Table 7: Base-model quantisation (LoRA $r = 8$, all linear layers, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Q-Bit	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
LLaMA-3.1-8B-Ins	16-bit	0.918	0.038	0.032	1.119	0.766	0.739	0.741	0.612
	8-bit	0.913	0.040	0.038	1.180	0.766	0.737	0.735	0.598
	4-bit	0.903	0.045	0.045	1.202	0.741	0.713	0.718	0.577
DeepSeek-R1-7B	16-bit	0.873	0.048	0.063	1.365	0.630	0.630	0.615	0.382
	8-bit	0.865	0.060	0.070	1.414	0.625	0.622	0.609	0.373
	4-bit	0.843	0.060	0.073	1.476	0.619	0.600	0.596	0.369
Qwen2.5-7B-Ins	16-bit	0.915	0.038	0.035	1.136	0.734	0.719	0.719	0.551
	8-bit	0.903	0.040	0.040	1.161	0.723	0.706	0.701	0.530
	4-bit	0.883	0.047	0.048	1.340	0.705	0.705	0.686	0.498

Table 7 compares 16-bit LoRA, 8-bit LLM.int8 and 4-bit NF4 QLoRA, with all other settings at the released default. The 16-bit rows come from separate training runs from those in Table 1.

For `LLaMA-3.1-8B-Instruct`, quantisation introduces only modest performance degradation. The F1-score declines from 0.741 at 16-bit to 0.735 at 8-bit and 0.718 at 4-bit, while Acc-P decreases from 0.918 to 0.903 under 4-bit quantisation. `Qwen2.5-7B-Instruct` follows a similar pattern, with F1-score declining from 0.719 to 0.701 at 8-bit and 0.686 at 4-bit. For both models, the F1-score under 4-bit NF4 stays within about 3 points of 16-bit, and the 8-bit loss is small (0.006 and 0.018). `DeepSeek-R1-7B` loses 0.019 in F1-score at 4-bit (0.615 to 0.596) and 0.030 in Acc-P.

The training resource requirements corresponding to these configurations are reported in Table 14 in Appendix B. For training, 4-bit quantisation reduces peak GPU memory by 47% for Llama (21.10 to 11.28 GB), but only by 25% for Qwen (21.37 to 15.95 GB) and 14% for DeepSeek (22.01 to 18.91 GB). Quantised training is also slower, by 9–19% at 4-bit and 16–28% at 8-bit.

Table 8: Inference cost under base-model quantisation (configuration as in Table 7).

		Costs					
Model	Q-Bit	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
LLaMA-3.1-8B-Ins	16-bit	15.6	15.6	9.0	4.0	21.7	0.113
	8-bit	9.3	9.3	17.4	7.1	45.8	0.057
	4-bit	8.2	8.1	11.4	4.9	25.2	0.088
DeepSeek-R1-7B	16-bit	14.8	14.8	8.6	3.7	26.7	0.116
	8-bit	11	9.9	18.2	6.8	55	0.055
	4-bit	7.7	7.7	10.2	4.4	25.3	0.0976
Qwen2.5-7B-Ins	16-bit	14.8	14.8	8.3	1.7	22.6	0.112
	8-bit	9.6	9.2	18.5	3.5	45.6	0.054
	4-bit	7.7	7.7	11.8	0.7	81.2	0.085

At inference (Table 8), peak GPU memory decreases with the bit width for all three models; for Llama it falls from 15.6 GB at 16-bit to 9.3 GB at 8-bit and 8.2 GB at 4-bit (40% and 47% reductions). Latency does not follow the same ordering. With the bitsandbytes kernels used here, LLM.int8 roughly doubles the average time per batch compared to 16-bit, while 4-bit NF4 is 19–42% slower than 16-bit but clearly faster than 8-bit.

6.2.3 Combined Configuration

Table 9: Best configuration per model found on the test split (optimistic; see Section 5.3), at 16-bit and 4-bit.

		Proactive Predictions				Tool Calling			
Model	Config	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	**0.942**	**0.02**	**0.023**	**0.943**	0.805	**0.800**	**0.789**	**0.682**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	0.93	0.038	0.028	0.987	**0.81**	0.791	0.782	0.673
DeepSeek-R1-7B	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 16-bit	**0.91**	**0.042**	**0.047**	**1.262**	**0.739**	**0.739**	**0.723**	**0.531**
	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 4-bit	0.878	0.053	0.065	1.297	0.671	0.667	0.653	0.446
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	**0.941**	**0.03**	0.033	**0.936**	**0.799**	**0.783**	**0.776**	0.650
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	0.928	0.045	**0.028**	1.057	0.792	0.770	0.765	**0.656**

Table 9 combines the best settings found above for each model. For Llama and Qwen this is $r = 32$, learning rate 10^{-4} and all linear layers; for DeepSeek it is $r = 64$, learning rate 2×10^{-5} and MLP layers only. For Llama and Qwen, the combined configuration is hardly better than the paper's configuration ($r = 8$, 10^{-4}; F1-score 0.789 versus 0.781 and 0.776 versus 0.779), so most of the improvement over the released default comes from the learning rate. For DeepSeek, the combined configuration reaches 0.723 against 0.689. The 4-bit version of the combined configuration stays within 0.011 F1-score of the 16-bit version for Llama (0.782 versus 0.789) and Qwen (0.765 versus 0.776), so 4-bit models with a suitable learning rate outperform the released 16-bit configuration (0.735 and 0.708) at about half the inference memory. DeepSeek loses 0.070 in F1-score at 4-bit with this configuration.

Training costs for these configurations are in Table 15 (Appendix B). At 4-bit, peak training memory falls from 21.63 to 12.24 GB for Llama, from 21.91 to 15.26 GB for Qwen and from 20.52 to 16.14 GB for DeepSeek, with training time increasing by at most 10%.

Table 10: Inference cost of the combined configurations.

		Costs					
Model	Config	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Llama-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	15.6	15.6	**9.1**	**4.2**	**21.7**	**0.110**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**8.2**	**8.2**	13.9	5.5	33.9	0.078
DeepSeek-R1-7B	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 16-bit	14.8	14.8	**8.7**	**3.3**	**20.5**	**0.115**
	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 4-bit	**7.7**	**7.7**	8.8	3.6	22.1	0.114
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	14.8	14.8	**8.7**	**3.8**	**22.5**	**0.115**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**7.7**	**7.7**	11.3	4.4	29.2	0.089

At inference (Table 10), 4-bit quantisation reduces peak GPU memory by 47–48% for all three models. For Llama and Qwen this comes with a 53% and 30% increase in average batch time. For DeepSeek, the average batch time is almost unchanged (8.8 versus 8.7 s).

6.2.4 Pruning

Pruning experiments use `Llama-3.1-8B-Instruct` with the released configuration; all results are after LoRA-based recovery fine-tuning. The dense reference is the first row of Table 1 (F1-score 0.735, Acc-Args 0.595).

Table 11: Unstructured and structured pruning of `Llama-3.1-8B-Instruct` after recovery fine-tuning.

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Unstructured	Wanda	10	**0.918**	**0.040**	**0.040**	**1.122**	**0.766**	**0.756**	**0.735**	**0.591**
		30	0.915	0.045	0.040	1.125	0.753	0.746	0.721	0.566
		50	0.910	0.047	0.042	1.137	0.735	0.722	0.717	0.541
	Magnitude	10	0.915	0.042	0.040	1.125	0.762	0.747	0.735	0.581
		30	0.910	0.048	0.043	1.134	0.747	0.742	0.731	0.567
		50	0.900	0.050	0.050	1.226	0.726	0.721	0.705	0.525
Structured	Activation	10	**0.870**	**0.082**	**0.047**	**1.371**	**0.651**	**0.651**	**0.637**	**0.409**
		30	0.717	0.172	0.110	1.944	0.493	0.490	0.481	0.235
		50	0.522	0.315	0.162	2.516	0.314	0.309	0.310	0.047
	Magnitude	10	0.470	0.530	0.000	3.407	0.467	0.467	0.467	0.000
		30	0.470	0.530	0.000	3.407	0.467	0.467	0.467	0.000
		50	0.470	0.530	0.000	3.407	0.467	0.467	0.467	0.000

Table 11 shows that unstructured pruning preserves performance much better than structured pruning. With WANDA, Acc-P stays above 0.90 and the F1-score at or above 0.717 even at 50% sparsity, against 0.735 for the dense model; Acc-Args drops from 0.595 to 0.541. Magnitude pruning behaves similarly at 10–30% and degrades slightly more at 50%. Structured activation-based pruning loses 0.10 in F1-score already at 10% and collapses at 50%. Structured magnitude pruning yields identical results at all three ratios, with MD = 0.53 and FD = 0: the model never predicts that assistance is needed. Its Acc-P and F1-score of 0.47 therefore correspond to the share of non-proactive samples, whose empty tool set is trivially matched. This is a degenerate solution rather than a gradual degradation.

Training costs (Table 16, Appendix B) show that unstructured pruning does not reduce recovery training time or memory, whereas 50% structured pruning reduces training time by about 48% and peak memory by 43% compared to 10% structured pruning.

Table 12: Inference cost after pruning and recovery.

			Costs					
Structure	Method	%	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Unstructured	Wanda	10	15.6	15.6	9.3	4.0	21.5	0.1077
		30	15.6	15.6	9.1	4.0	21.8	0.1103
		50	15.6	15.6	9.2	3.8	23.8	0.1087
	Magnitude	10	15.6	15.6	9.2	4.1	27.6	0.1092
		30	15.6	15.6	8.9	3.9	21.2	0.1123
		50	15.6	15.6	9.1	3.9	33.3	0.1093
Structured	Activation	10	13.2	13.2	8.2	3.7	17.8	0.1220
		30	8.9	8.9	5.4	2.3	13.3	0.1850
		50	5.7	5.6	5.3	1.5	32.1	0.1883
	Magnitude	10	13.3	13.3	34.8	4.6	62.0	0.0287
		30	9.0	9.0	43.7	42.6	47.6	0.0229
		50	5.7	5.6	31.4	28.2	41.4	0.0318

At inference (Table 12), unstructured pruning leaves memory and latency unchanged because the pruned weights are stored and multiplied densely. Structured magnitude pruning is 3.5–5 times *slower* than the dense model despite its smaller size. This slowdown coincides with degenerate generation: for the 10%, 30% and 50% settings, all 400 evaluated samples produced repetitive, malformed outputs that failed to yield parseable

plans. The inference configuration allowed up to 1,024 newly generated tokens; however, per-sample token counts were not recorded, so reaching the exact generation limit cannot be confirmed.

7 Discussion

7.1 What Reproduces and What Does Not

The qualitative claims of *ContextAgent* hold: fine-tuning with persona context and thought traces outperforms in-context learning for all three base models (C1), and personas plus thought traces improve in-context learning (C2). The in-context baselines also reproduce numerically. The fine-tuned results do not: with the released code, two of three models score clearly higher than reported, and with the learning rate stated in the paper all three do (C3).

Two under-specified details make this difference hard to attribute. First, the paper and the released configuration disagree on the learning rate (10^{-4} versus 5×10^{-5}), and the choice between these two values alone changes tool-calling F1 by 0.05–0.08 for the same model (Table 6). Second, the paper reports eight GPUs, but not whether training was data-parallel or what the resulting effective batch size was. With only 600 training samples, an eight-fold difference in batch size changes the number of optimiser steps from about 375 to about 50, which may leave the original models undertrained; this is consistent with both the higher scores we obtain and the different error profile (MD versus FD) of the reported models. For future users of *ContextAgentBench*, we recommend reporting the effective batch size and number of optimiser steps, and treating the learning rate as a tuned hyperparameter selected on a held-out part of the training split.

A practical consequence is that the efficiency results below are measured against a stronger baseline than the paper reports: an efficiency technique that "preserves the reported performance" may still lose a substantial part of the performance the released code can reach.

7.2 Efficiency of the Proactive Reasoner

LoRA configuration. Since *ContextAgent* already uses LoRA, the relevant question is which configuration is sufficient. For the instruction-tuned models, rank 8 on all linear layers is enough once the learning rate is adequate; higher ranks add little (Table 9). DeepSeek-R1-7B benefits from higher rank and from restricting adaptation to MLP layers. A possible explanation is that this reasoning-oriented model needs more adapter capacity to reconcile its long free-form reasoning with the strict JSON output format, and that adapting attention layers interferes with its reasoning behaviour; we have not tested these explanations directly.

Quantisation. Base-model quantisation reduces the dominant memory cost, the frozen weights, at both training and inference time. For Llama and Qwen, 4-bit NF4 costs at most 0.011 in tool-calling F1 at the combined configuration and about 0.03 at the default, while halving inference memory. DeepSeek is more sensitive. The latency penalty, roughly double for LLM.int8 and up to 53% for NF4, is a property of the bitsandbytes kernels, which dequantise weights on the fly. It should not be read as an inherent cost of 4-bit inference: post-training quantisation with dedicated kernels (Activation-aware Weight Quantisation (AWQ), post-training quantisation for Generative Pre-trained Transformers (GPTQ)) or on-device runtimes would likely change this picture. For a proactive agent that evaluates context continuously, the memory reduction is what allows a 7–8B reasoner to run on a single consumer GPU or edge accelerator, but the latency increase directly lengthens the decision loop.

Pruning. Unstructured and semi-structured pruning preserve the task metrics up to 50% sparsity but bring no benefit with dense kernels. Structured pruning of MLP dimensions brings real memory and latency savings, but the task degrades quickly; with magnitude-based group importance, the model collapses to never predicting a need for assistance. For a proactive agent, this failure mode is particularly harmful because it is silent: the agent appears to work but never acts. Activation-based importance degrades gradually, which indicates that data-dependent criteria are needed for structured pruning in this setting.

Cost–performance trade-off. Among the techniques tested, 4-bit quantisation of the base model with a well-chosen LoRA configuration offers the best trade-off: it matches or exceeds the released 16-bit model at about half the inference memory. Structured pruning beyond 10% and unstructured pruning without sparse kernels do not provide a useful trade-off in our setting.

7.3 Limitations

Test-set selection and single-run evaluation. The released split has no validation set, so all configuration sweeps are evaluated on the test split. The "best configuration" results should therefore be interpreted as optimistic upper bounds from a sensitivity sweep rather than a tuned model selection. In contrast, the reproduction results for the released and paper configurations (Table 2) evaluate the specified configurations directly without hyperparameter selection. Furthermore, each setting is evaluated on a single completed training run ($k = 1$), meaning between-run variance cannot be estimated. With 400 test samples, differences of a few hundredths in a single metric should not be over-interpreted.

Learning-rate grid. The largest learning rate tested, 10^{-4}, is the best for two models, so the optimum may lie outside the grid. The released default 5×10^{-5} is the weakest of the three values for all models, which we cannot explain from our data.

Scope of cost measurements. Costs are measured for the reasoner only, with the bitsandbytes runtime on a single GPU. The perception stage (vision-language model and speech recognition), deployment runtimes and on-device measurements are out of scope.

Pruning. Pruning was evaluated on `Llama-3.1-8B-Instruct` only, structured pruning was restricted to MLP dimensions because preliminary experiments with attention-head pruning degraded the model severely, and recovery used LoRA rather than full fine-tuning.

Knowledge distillation (negative result). We also attempted response-based distillation from each 7–8B model into a 1–1.5B student of the same family (Llama-3.2-1B, DeepSeek-R1-1.5B, Qwen2.5-1.5B), with a combined cross-entropy and Kullback–Leibler objective (temperature 3, $\alpha = 0.5$), 4-bit NF4 quantisation, and a maximum sequence length of 1024 tokens to fit both models into 24 GB. None of the students learned to produce the required JSON output, so no task metrics could be computed. The most likely cause is truncation: *ContextAgent* prompts with tool definitions, persona and context regularly exceed 1024 tokens, so the students rarely saw complete instructions and targets. We therefore do not draw conclusions about distillation for proactive agents; a fair test would first fine-tune the student on the full-length data and distil from cached teacher outputs.

7.4 Future Work

Proactive agents spend most of their time deciding *not* to act. A natural efficiency direction specific to this setting is to emit the proactive score before the thought trace and stop generation when it falls below the threshold, or to gate the full reasoner with a small model trained only on the proactive score. Measuring cost per hour of monitoring rather than per sample, and evaluating post-training quantisation kernels and on-device runtimes, would make the efficiency conclusions directly applicable to deployment.

8 Conclusion

We reproduced *ContextAgent* on *ContextAgentBench* with the authors' released artefacts. The central claims hold: supervised fine-tuning with personas and thought traces clearly outperforms in-context learning, and the in-context baselines reproduce closely. The fine-tuned models, however, reach substantially higher scores than reported, and the size of the gap depends on a learning-rate discrepancy between the paper and the released code. Users of the benchmark should therefore treat the reported fine-tuned scores as a lower bound and report the learning rate, effective batch size and number of optimiser steps.

Under resource constraints, 4-bit quantisation of the base model combined with an adequate LoRA configuration retains near-16-bit performance for the two instruction-tuned models at half the inference memory, at the cost of higher latency with on-the-fly dequantisation. Unstructured pruning preserves performance without efficiency gains on dense hardware, and structured pruning yields efficiency gains only at a large and sometimes silent loss of proactive behaviour. Efficient proactive agents therefore need methods that reduce cost while preserving the decision to act and the structured output required to do so.

Code availability

Code, configuration files and raw evaluation outputs are available at an anonymised repository for review: anonymized: https://anonymous.4open.science/r/tmlr-sub-eval/ (Camera-ready: `https://github.com/DAT8-CS-IT/revisiting-contextagent`.)

References

Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations, 2024. URL https://arxiv.org/abs/2308.06767.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL https://arxiv.org/abs/2208.07339.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

Christopher Diebel, Marc Goutier, Martin Adam, and Alexander Benliar. When ai-based agents are proactive: Implications for competence and system satisfaction in human–ai collaboration, 2024. URL https://doi.org/10.1007/s12599-024-00918-y.

Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. Depgraph: Towards any structural pruning, 2023. URL https://arxiv.org/abs/2301.12900.

Beakcheol Jang Gun Il Kim, Sunga Hwang. Efficient compressing and tuning methods for large language models: A systematic literature review, 2025. URL https://doi.org/10.1145/3728636.

Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531, 2015. URL http://arxiv.org/abs/1503.02531.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting llm agents from reactive responses to active assistance, 2024. URL https://arxiv.org/abs/2410.12361.

Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks, 2021. URL https://arxiv.org/abs/2104.08378.

Waldir S. S. Junior Pierre V. Dantas, Lucas C. Cordeiro. A review of state-of-the-art techniques for large language model compression, 2025. URL https://doi.org/10.1007/s40747-025-02019-z.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models, 2024. URL https://arxiv.org/abs/2306.11695.

Bufang Yang, Lilin Xu, Liekang Zeng, Yunqi Guo, Siyang Jiang, Wenrui Lu, Kaiwei Liu, Hancheng Xiang, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. Proagent: Harnessing on-demand sensory contexts for proactive llm agent systems, 2025a. URL https://arxiv.org/abs/2512.06721v1.

Bufang Yang, Lilin Xu, Liekang Zeng, Kaiwei Liu, Siyang Jiang, Wenrui Lu, Hongkai Chen, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. Contextagent: Context-aware proactive llm agents with open-world sensory perceptions, 2025b. URL https://arxiv.org/abs/2505.14668v2.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

A Acronyms

AWQ Activation-aware Weight Quantisation

Acc-Args Argument Accuracy

Acc-P Proactive Prediction Accuracy

FD False Detection rate

GPTQ post-training quantisation for Generative Pre-trained Transformers

KD Knowledge Distillation

LLM Large Language Model

LoRA Low-Rank Adaptation

MD Missed Detection rate

MLP Multilayer Perceptron

NF4 4-bit NormalFloat

QLoRA Quantised Low-Rank Adaptation

RMSE Root Mean Square Error

SFT Supervised Fine-Tuning

WANDA Pruning by **W**eights **and** **A**ctivations

B Training Costs

Table 13: Training cost of the reproduced *ContextAgent* models (released configuration).

		Costs		
Method	Model	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
ContextAgent	Llama-3.1-8B-Ins	2691.0	21.02	17.79
	DeepSeek-R1-7B	3446.6	21.02	16.68
	Qwen2.5-7B-Ins	2862.2	21.35	17.08

Table 14: Training cost under base-model quantisation (LoRA $r = 8$, learning rate 5×10^{-5}).

		Costs		
Model	Q-Bit	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	16-bit	2855.8	21.10	18.94
	8-bit	3656.8	19.07	18.75
	4-bit	3248.1	11.28	10.77
DeepSeek-R1-7B	16-bit	2921.8	22.01	17.17
	8-bit	3398.6	19.95	13.6
	4-bit	3474.7	18.91	16.15
Qwen2.5-7B-Ins	16-bit	2935.6	21.37	17.31
	8-bit	3609.9	19.95	13.9
	4-bit	3211.3	15.95	15.18

Table 15: Training cost of the combined configurations.

		Costs		
Model	Config	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	3006.5	21.63	18.67
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	3305.7	12.24	11.73
DeepSeek-R1-7B	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 16-bit	2761.7	20.52	19.53
	$r = 64, lr = 2.0 \times 10^{-5}$, MLP, 4-bit	2964.5	16.14	15.09
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	2985.1	21.91	18.23
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	2996.5	15.26	14.69

Table 16: Recovery training cost after pruning (`Llama-3.1-8B-Instruct`).

			Costs		
Structure	Method	%	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Unstructured	Wanda	10	2787.0	19.54	18.95
		30	2770.5	19.54	18.90
		50	2765.2	19.54	18.87
	Magnitude	10	2782.0	19.54	18.79
		30	2751.5	19.54	18.93
		50	2710.2	19.54	18.97
Structured	Activation	10	2891.5	19.71	18.84
		30	1986.3	15.33	14.55
		50	1479.3	11.14	10.64
	Magnitude	10	2745.0	19.71	18.83
		30	1926.3	15.33	14.43
		50	1524.0	11.14	10.36

C Semi-structured Pruning

Semi-structured pruning aims to balance the flexibility of unstructured pruning with the hardware efficiency of structured sparsity. A widely adopted formulation is **N:M** sparsity, in which exactly N non-zero weights are retained within each contiguous block of M parameters (Mishra et al., 2021). This specific pattern is highly relevant for modern hardware accelerators, such as GPUs equipped with Sparse Tensor Cores, which natively accelerate 2:4 sparsity to achieve tangible inference speedups.

Table 17: Semi-structured pruning (50% sparsity) of `Llama-3.1-8B-Instruct` after recovery fine-tuning.

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Semi-structured	Wanda	2:4 (50%)	0.890	0.050	0.060	1.281	0.670	0.708	0.687	0.505
		4:8 (50%)	**0.910**	**0.047**	**0.040**	**1.203**	**0.715**	**0.730**	**0.719**	**0.535**
	Magnitude	2:4 (50%)	0.888	0.072	0.040	1.295	0.705	0.715	0.694	0.510
		4:8 (50%)	0.888	0.070	0.042	1.269	0.713	0.718	0.701	0.53

Table 17 shows the task performance of semi-structured pruning at 50% sparsity with 2:4 and 4:8 patterns. The less constrained 4:8 pattern performs at least as well as 2:4 for both criteria; for WANDA it reaches an F1-score of 0.719 and Acc-Args of 0.535, against 0.687 and 0.505 for 2:4, and it matches unstructured WANDA at the same sparsity (0.717). With the 4:8 pattern, WANDA is slightly better than magnitude pruning (F1-score 0.719 versus 0.701); with 2:4, the two criteria are indistinguishable. All semi-structured configurations keep Acc-P above 0.88 and the F1-score above 0.68.

Table 18: Recovery training cost after semi-structured pruning.

			Costs		
Structure	Method	%	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Semi-structured	Wanda	2:4 (50%)	2719.8	19.54	18.82
		4:8 (50%)	2732.5	19.54	18.91
	Magnitude	2:4 (50%)	2700.6	19.54	18.96
		4:8 (50%)	2726.3	19.54	18.89

Training costs are reported in Table 18. Peak GPU memory is 19.54 GB for all configurations, as for unstructured pruning, and training times range from 2701 to 2733 seconds.

Table 19: Inference cost after semi-structured pruning.

			Costs					
Structure	Method	%	GPU Peak (GB)↓	GPU Avg (GB)↓	Avg Batch (s)	Min Batch (s)	Max Batch (s)	Avg Samples/s
Semi-structured	Wanda	2:4 (50%)	15.6	15.6	9.1	3.8	27.2	0.1100
		4:8 (50%)	15.6	15.6	9.1	3.8	23.5	0.1088
	Magnitude	2:4 (50%)	15.6	15.6	9.0	3.7	22.0	0.1109
		4:8 (50%)	15.6	15.6	8.8	3.0	21.1	0.1133

At inference (Table 19), peak GPU memory remains 15.6 GB and the average batch time 8.8–9.1 s, the same as the dense model. Exploiting 2:4 sparsity requires sparse tensor-core kernels, which our inference stack did not use.